基于 Pytorch 的深度学习实践 (二)

YYFL

March 2026

1 反向传播 (Backpropagation)

1.1 核心定义

反向传播是深度学习中训练神经网络的核心算法，本质是**利用链式法则从损失函数出发，反向计算每个参数的梯度**，并通过梯度下降更新参数，最终最小化损失函数。其核心思想是：

$$\text{损失函数对参数的梯度} = \text{损失对输出的梯度} \times \text{输出对参数的梯度（链式法则）}$$

1.2 单参数反向传播（基础入门）

1.2.1 网络结构

单层线性神经网络：输入 $x \to$ 权重 $w \to$ 输出 $y \to$ 损失 L
数学定义：

$$y = w \cdot x$$

$$L = (y - \text{target})^2$$

1.2.2 具体数值

$x = 2,\ w = 3,\ \text{target} = 10$

1.2.3 前向传播（Forward Pass）

计算预测值和损失：

$$y = w \cdot x = 3 \times 2 = 6$$

$$L = (y - \text{target})^2 = (6 - 10)^2 = 16$$

1.2.4 反向传播（Backward Pass）

目标：计算 $\frac{\partial L}{\partial w}$（损失对权重的梯度），判断权重调整方向。

链式法则（核心） L 间接依赖 w（$L \leftarrow y \leftarrow w$），因此：

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial w}$$

分步计算

1. 计算 $\frac{\partial L}{\partial y}$：
$$\frac{\partial L}{\partial y} = 2 \times (y - \text{target}) = 2 \times (6 - 10) = -8$$
2. 计算 $\frac{\partial y}{\partial w}$：
$$\frac{\partial y}{\partial w} = x = 2$$
3. 链式法则相乘：
$$\frac{\partial L}{\partial w} = -8 \times 2 = -16$$

1.2.5 梯度更新

权重更新公式：
$$w_{\text{new}} = w_{\text{old}} - \eta \times \frac{\partial L}{\partial w}$$
设学习率 $\eta = 0.1$：
$$w_{\text{new}} = 3 - 0.1 \times (-16) = 4.6$$
更新后验证：$y = 4.6 \times 2 = 9.2$，损失降低为 $(9.2 - 10)^2 = 0.64$，验证有效。

1.3 多参数反向传播（含偏置项）

1.3.1 网络结构

单层线性网络（含偏置 b）：
$$y = w \cdot x + b$$
$$L = (y - \text{target})^2$$

1.3.2 具体数值

$x = 2,\ w = 3,\ b = 1,\ \text{target} = 10$

1.3.3 前向传播

$$y = 3 \times 2 + 1 = 7$$
$$L = (7 - 10)^2 = 9$$

1.3.4 反向传播

目标：计算 $\frac{\partial L}{\partial w}$ 和 $\frac{\partial L}{\partial b}$

1. $\frac{\partial L}{\partial y} = 2 \times (7 - 10) = -6$
2. $\frac{\partial y}{\partial w} = x = 2$，$\frac{\partial y}{\partial b} = 1$
3. 梯度计算：
$$\frac{\partial L}{\partial w} = -6 \times 2 = -12$$
$$\frac{\partial L}{\partial b} = -6 \times 1 = -6$$

1.3.5 梯度更新

设 $\eta = 0.1$：

$$w_{\text{new}} = 3 - 0.1 \times (-12) = 4.2$$

$$b_{\text{new}} = 1 - 0.1 \times (-6) = 1.6$$

更新后：$y = 4.2 \times 2 + 1.6 = 10$，损失为 0，完美拟合。

1.4 多层神经网络的反向传播（以 2 层为例）

1.4.1 网络结构

两层全连接网络（含 ReLU 激活）：

$$z_1 = w_1 \cdot x + b_1$$

$$a_1 = \sigma(z_1) \quad (\sigma = \text{ReLU})$$

$$z_2 = w_2 \cdot a_1 + b_2$$

$$y = z_2$$

$$L = (y - \text{target})^2$$

1.4.2 链式法则扩展

梯度需逐层反向计算：

- 输出层（第二层）：$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial z_2} \times \frac{\partial z_2}{\partial w_2}$
- 隐藏层（第一层）：$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial z_2} \times \frac{\partial z_2}{\partial a_1} \times \frac{\partial a_1}{\partial z_1} \times \frac{\partial z_1}{\partial w_1}$

1.4.3 ReLU 激活函数的梯度

$$\sigma(z) = \max(0, z), \quad \sigma'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

1.4.4 示例计算

设 $x = 1,\ w_1 = 2,\ b_1 = 1,\ w_2 = 3,\ b_2 = 2,\ \text{target} = 15$：

- 前向传播：$z_1 = 2 \times 1 + 1 = 3 \to a_1 = \text{ReLU}(3) = 3$ $z_2 = 3 \times 3 + 2 = 11 \to y = 11$ $L = (11 - 15)^2 = 16$
- 反向传播：$\frac{\partial L}{\partial y} = 2 \times (11 - 15) = -8$ $\frac{\partial y}{\partial z_2} = 1$ $\frac{\partial z_2}{\partial w_2} = a_1 = 3 \to \frac{\partial L}{\partial w_2} = -8 \times 1 \times 3 = -24$ $\frac{\partial z_2}{\partial a_1} = w_2 = 3,\ \frac{\partial a_1}{\partial z_1} = 1,\ \frac{\partial z_1}{\partial w_1} = x = 1$ $\frac{\partial L}{\partial w_1} = -8 \times 1 \times 3 \times 1 \times 1 = -24$

1.5 反向传播的代码实现（手动 vs 自动微分）

1.5.1 手动实现反向传播（理解原理）

```python
import torch

# 手动设置参数（禁用自动微分）
x = torch.tensor(2.0)
w = torch.tensor(3.0)
b = torch.tensor(1.0)
target = torch.tensor(10.0)

# 前向传播
y = w * x + b
loss = (y - target) ** 2

# 手动反向传播计算梯度
dL_dy = 2 * (y - target)
dy_dw = x
dy_db = 1.0
dL_dw = dL_dy * dy_dw
dL_db = dL_dy * dy_db

print(f"手动计算 dL/dw = {dL_dw.item()}")  # 输出 -12.0
print(f"手动计算 dL/db = {dL_db.item()}")  # 输出 -6.0
```

1.5.2 PyTorch 自动微分（Autograd）

```python
import torch

# 开启自动微分
x = torch.tensor(2.0)
w = torch.tensor(3.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
target = torch.tensor(10.0)

# 前向传播（构建计算图）
y = w * x + b
loss = (y - target) ** 2

# 自动反向传播
loss.backward()

# 查看梯度
print(f"Autograd计算 dL/dw = {w.grad.item()}")  # 输出 -12.0
print(f"Autograd计算 dL/db = {b.grad.item()}")  # 输出 -6.0
```

1.6 反向传播的常见问题与解决

1.6.1 梯度消失（Vanishing Gradient）

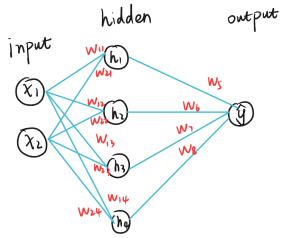
- 现象：深层网络中，输入层附近参数梯度趋近于 0，无法更新。
- 原因：Sigmoid 等激活函数梯度极值区趋近于 0，链式法则连乘后梯度消失。
- 解决：ReLU/Leaky ReLU 激活、残差连接、批归一化、Xavier/He 初始化。

1.6.2 梯度爆炸（Exploding Gradient）

- 现象：梯度值过大，参数更新幅度过大，训练不稳定（NaN/Inf）。
- 原因：链式法则连乘后梯度指数级增长。
- 解决：梯度裁剪、权重初始化、批归一化。

1.7 核心总结

1. 核心：链式法则，从损失反向计算每个参数的梯度；
2. 目的：找到参数最优更新方向，最小化损失；
3. 工具：PyTorch Autograd 自动实现，无需手动计算；
4. 挑战：梯度消失/爆炸，需通过激活函数、残差连接等优化。



$$\sigma\left((x_1, x_2)\begin{pmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \end{pmatrix}\right) = (h_1, h_2, h_3, h_4)$$

$(1,2)(2,4)$ $\qquad$ $(1,4)$

隐藏层 = 线性变换 + 激活函数

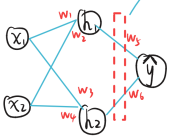
$$\boxed{\begin{array}{l} \text{nn.Linear}(764, 128) \\ \text{nn.ReLu}(\quad) \end{array}}$$ 线性层

$$\boxed{\begin{array}{l} \text{nn.Conv2d}(\quad) \\ \text{nn.Relu}(\quad) \\ \text{nn.MaxPooling2d}(\quad) \end{array}}$$ 卷积块

$$h_1 = \sigma(w_{11}x_1 + w_{21}x_2)$$
$$h_2 = \sigma(w_{12}x_1 + w_{22}x_2)$$
$$h_3 = \sigma(w_{13}x_1 + w_{23}x_2)$$
$$h_4 = \sigma(w_{14}x_1 + w_{24}x_2)$$

反向传播

省略了激活函数



假设 input $= (x_1, x_2) = (0.5, 1.0)$

标签 $y = 0.8$

随机初始化 $w_1 = 1.0$, $w_2 = 0.5$, $w_3 = 0.5$, $w_4 = 0.7$, $w_5 = 1.0$, $w_6 = 2.0$

$$\boxed{\begin{array}{l} h_1 = w_1x_1 + w_2x_2 = 1.0 \\ h_2 = w_1x_3 + w_3x_4 = 0.95 \\ \hat{y} = w_5h_1 + w_6h_2 = 2.9 \\ \text{loss} = \frac{1}{2}(y-\hat{y})^2 = 2.205 \end{array}}$$

正向传播 1

backpropagation

迭代更新每一个参数

$$\frac{\partial \text{loss}}{\partial w_5} = \frac{\partial \text{loss}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_5}$$
$$= (\hat{y} - y) \cdot h_1$$
$$= 2.1$$

更新 w_5

学习率

$$w_5 = w_5 - \eta \frac{\partial \text{loss}}{\partial w_5}$$
$$= 1.0 - 0.1 \times 2.1 = 0.79$$

$$\frac{\partial \text{loss}}{\partial w_1} = \frac{\partial \text{loss}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$
$$= (\hat{y} - y) \cdot w_5 \cdot x_1$$
$$= 2.1 \times 1.0 \times 0.5 = 1.05$$

更新 w_1

$$w_1 = w_1 - \eta \frac{\partial \text{loss}}{\partial w_1} = 1.0 - 0.1 \times 1.05$$
$$= 0.895$$

同理更新 $w_2 = 0.29$, $w_3 = 0.29$, $w_4 = 0.28$, $w_6 = 1.8005$

则第1轮反向传播结束

$$\boxed{\begin{array}{l} h_1 = w_1x_1 + w_2x_2 = 0.7375 \\ h_2 = w_3x_1 + w_4x_2 = 0.425 \\ \hat{y} = w_5h_1 + w_6h_2 = 1.3478 \\ \text{Loss} = \frac{1}{2}(y-\hat{y})^2 = 0.15 \end{array}}$$

正向传播 2

图 1: 反向传播梯度推导